

Citadel — Intent Mandate Enforcement (open demo)

Security & Design Report

Generated 2026-06-22

1. Executive summary

`citadel-mandate-demo` is a small, standalone, open (Apache-2.0) Rust crate that demonstrates **runtime enforcement of user-signed AP2-style Intent Mandates** for AI-agent payments. An AI agent acting for a user can only spend within the mandate its user cryptographically signed; anything out of scope is denied *before* a payment could happen.

This report describes precisely **what the demo is and is not, how it works**, and the result of a **comprehensive adversarial test** of it.

Test verdict. Within its stated scope (verify Ed25519 signature → check scope/limits → allow + token *or* deny + audit), the enforcement is **not bypassable** and does **not panic or crash on attacker-controlled input**. This is backed by **27 passing tests — 25 of them active attack attempts** — plus an independent multi-agent source audit. Every “weakness” found is a **documented, by-design simplification** of a mechanism that the full (separate) Citadel platform implements in hardened form — not a defect in the demo’s security core.

Dimension	Result
Signature forgery / tampering (12 vectors)	All denied
Scope / limit evasion (cap, merchant, currency, expiry)	All denied
Malformed / huge / truncated input	Denied, no panic
Decision integrity (deny never mints a token)	Holds
Audit coverage (allow <i>and</i> deny recorded; chain verifies)	Holds
Exploitable bypass that yields an unintended Allow	None found

2. What it is — and what it is not

It is

- A **runtime enforcement layer**: it verifies a user’s signed mandate and decides allow/deny on a concrete charge, fail-closed.

- **Intent Mandate only** — “agent X may spend up to \$N at these merchants in this currency until time T.”
- A faithful reimplementation of the **real verification algorithm** (`parse` → `canonical` `serialize` → `SHA-256` → `Ed25519` `verify_strict`), fail-closed on every error path.
- Standalone: **zero dependency on any private/monorepo code**; builds and runs with public crates only.

It is not

- **Not** full **W3C Verifiable Credential** or AP2 wire interop. It uses *VC-style* Ed25519 detached signatures, not the full VC/JOSE envelope.
- **Not** a Cart or Payment Mandate engine (Intent only).
- **Not** a payment processor — it holds no money and contacts no PSP. It is the *decision* + *proof* layer.
- The **audit log** and **token** here are **simplified illustrations**. The production system adds a Merkle-tree audit chain with **hybrid post-quantum** signatures (Ed25519 + ML-DSA-65) and **RFC 9449 enclave-bound DPOP**.
- **Not** the platform: mutual-TLS identity, a policy engine, spend limits, multi-tenancy, kill-switch, SIEM/RBAC/HSM, and the managed gateway are a separate product.

3. How it works

3.1 The mandate

An **Intent Mandate** is a JSON document the user signs with their Ed25519 key:

```
{
  "mandate_id": "im_demo_0001",
  "user_id": "alice@example.com",
  "agent_id": "agent-7c3f-shopping",
  "intent_description": "weekly groceries",
  "max_amount_cents": 15000,
  "currency": "usd",
  "allowed_merchants": ["acme-store.example"],
  "created_at": "2026-01-01T00:00:00Z",
  "expires_at": "2030-01-01T00:00:00Z",
  "signature_b64": "..." // detached Ed25519 over the canonical mandate (sans signature)
}
```

The signature covers **every field except** `signature_b64` **itself**. The signed bytes are `SHA-256(serde_json(mandate))` over the fixed-field-order struct, so the signer and verifier hash identical bytes regardless of wire key ordering.

3.2 The enforcement pipeline

```
signed mandate + charge
|
▼ verify_signed    → resolve user_id → trusted key (unknown ⇒ DENY)
                    Ed25519 verify_strict over SHA-256(payload) (bad ⇒ DENY)
|
▼ check_scope      → expiry · currency · amount ≤ cap · merchant ∈ allowlist
                    (any violation ⇒ DENY)
|
▼ ALLOW            → mint action-bound, expiring Ed25519 token
|
every outcome (ALLOW and DENY) → one hash-chained audit entry
```

- **Trusted-key registry** — a server-seeded `user_id → Ed25519 public key` map. **Empty registry = every mandate denied** (fail-closed).
- **Fail-closed invariant** — every error, ambiguity, or violation returns a Deny. A Deny is produced *before* the token-minting branch, so a denied request can never receive a token.
- **Audit** — each decision appends `entry_hash = SHA-256(prev_hash || summary); verify()` recomputes the chain and detects any mutation or truncation.
- **Token** — minted **only on Allow**: an Ed25519-signed compact token binding the agent id, an action hash (merchant||amount||currency), a unique `jti`, and a 300-second expiry.

3.3 Interfaces

- **CLI** (`cargo run`): runs the 5 canonical scenarios and prints a cast-ready table.
- **HTTP** (`cargo run -- serve`): `POST /v1/authorize` with `{ mandate, charge }` returns `{ decision, reason?, token_id?, audit_seq, audit_head }`.
- **mint**: prints a ready-to-`curl` signed request (incl. tamper variants).

4. Threat model

- **Attacker controls**: the mandate wire JSON and the charge (merchant, amount, currency) submitted to the gateway.
- **Attacker does not control**: the server-seeded trusted-key registry, the gateway's token-signing key.
- **Goal of an attack**: make the gateway **Allow** a charge the signed mandate does not authorize (forge/alter a mandate, exceed the cap, hit a disallowed merchant, use an expired mandate), or **crash/DoS** the gateway with crafted input.

5. Adversarial test results

5.1 Executable attack suite (`tests/adversarial.rs` , 25 tests — all pass)

A. Signature / identity forgery (15) — all DENIED: unsigned mandate · empty registry vs a perfect signature · signed by the wrong key · raise cap after signing · add merchant after signing · swap `user_id` · swap `agent_id` · extend expiry after signing · garbage signature · non-base64 signature · signature lifted

from another mandate · low-level `verify_signed` forgery battery · **duplicate-key split-view** · **type-confusion (number as string)** · unknown extra field is inert (still verifies, grants nothing).

B. Scope / limit evasion (5) — boundaries correct: amount exactly at cap → Allow, cap + 1¢ → Deny · `u64::MAX` amount → Deny (no overflow) · currency mismatch → Deny · merchant outside allowlist → Deny · expired → Deny, in-window → Allow.

C. Robustness (2) — no panic, fail-closed: malformed/empty/ `null` / `[]` JSON → Deny · 2 MB junk and truncated wires → Deny.

D. Audit integrity (1): allow **and** deny each produce exactly one entry; the hash-chain verifies.

E. By-design behaviour, recorded (2): empty `allowed_merchants` permits any merchant within cap; the same mandate may be reused (no replay protection).

(plus `tests/enforcement.rs` : the 5 headline scenarios as hard assertions.)

5.2 Independent multi-agent source audit

An adversarial review was run over the source. The **cryptographic lens** returned a complete result and **empirically confirmed**, among others:

- `verify_strict` (not the malleable legacy `verify`) is used — a signature with `S' = S + L` (the canonical Ed25519 malleability transform) is **rejected**.
- **Every** scope/limit field is inside the signed payload; the only unsigned wire field is `signature_b64` itself.
- **Duplicate JSON keys** and **type-confusion** wires are **rejected** by `serde` (no signature-vs-enforcement split-view).
- `user_id` is both the key selector *and* a signed field ⇒ **no cross-user key confusion**; Ed25519 is fixed ⇒ no algorithm-downgrade vector.

The scope and robustness review agents were interrupted by a transient network error; those dimensions are instead covered by the executable suite (§5.1 B/C) and the code review in §6–7. Verdict (crypto lens): *“NOT bypassable in this demo.”*

6. Positive security properties

1. **Fail-closed everywhere** — unknown user, bad/missing/garbage signature, malformed wire, expired/over-cap/out-of-allowlist all return Deny.
2. **No attacker-triggerable panic** — the request path (`verify_signed` → `check_scope` → `authorize`, and the HTTP handler) maps every error; the only `expect` / `unwrap` calls are on the signer side or on gateway-built data, not on attacker input. Proven by the malformed/huge/truncated tests.
3. **Tamper-evident decisions** — every allow and deny is recorded; the audit hash-chain detects mutation/truncation.
4. **Allow-only token** — a token is minted strictly after scope passes; a Deny cannot carry one.

5. **Correct Ed25519 usage** — `verify_strict` blocks signature malleability and small-order/non-canonical encodings.

7. Limitations & out-of-scope (by design)

These are deliberate simplifications for a small open demo. Each maps to a hardened mechanism in the full Citadel platform. **None of them lets an attacker obtain an unintended Allow in the demo's threat model.**

#	Demo behaviour	Why it is acceptable here	Production counterpart
L1	Audit log is hash-chained but not signed	tamper-evident against mutation; demonstrates the idea	Merkle chain + hybrid PQ (Ed25519 + ML-DSA-65) signed tree-head
L2	Minted token is never verified inside the demo	shows issuance, not consumption	RFC 9449 DPoP , enclave-bound key, verified at the resource
L3	No replay / nonce / spend accumulation — a mandate is a standing authorization	keeps the demo stateless and readable	per-request nonce + stateful SpendTracker (daily/tx/rate caps)
L4	Empty <code>allowed_merchants</code> = any merchant	matches "Intent with no merchant restriction"	policy engine can require an explicit allowlist
L5	HTTP body has no custom size cap (Axum's ~2 MB default applies)	bounded by the framework default	explicit body limits + rate limiting + mTLS L0
L6	Gateway state is in-memory ; the shared lock can poison on panic	single-process demo; request path doesn't panic	persistent stores; non-poisoning concurrency
L7	Intent only; VC-style (not full W3C VC / AP2 interop)	the focused wedge	Cart/Payment mandates; full protocol adapters

8. How it maps to production

The verification algorithm here is the same one Citadel runs on its live decision path. What is stripped out for this demo is the **platform around the core**, not the security core itself:

```
demo:      verify(Ed25519) → scope/limit → hash-chain audit → Ed25519 token
production: mTLS L0 → parse → policy engine (L2) → spend limits (L3)
           → PQ-signed Merkle audit (CX) → enclave-bound DPoP (L4)
           - multi-tenant, fail-closed, with kill-switch / SIEM / RBAC / HSM
```

9. Reproduce

```
# native
cargo test          # 27 tests (25 adversarial + by-design + scenarios)
cargo run           # the 5-scenario cast

# container (verified in CI on Linux runners)
docker build -t citadel-mandate-demo . && docker run --rm citadel-mandate-demo

# HTTP gateway
cargo run -- serve
cargo run -- mint --over | curl -s localhost:8080/v1/authorize \
  -H 'content-type: application/json' -d @-
```

CI ([.github/workflows/ci.yml](#)) runs `fmt` + `clippy -D warnings` + `test` + the demo **and** `docker build` / `docker run` on clean Linux runners on every push.

10. Appendix — inventory

Source (785 lines): `mandate.rs` (135, verification core) · `main.rs` (123, CLI) · `scenarios.rs` (107) · `serve.rs` (104, HTTP) · `engine.rs` (89, pipeline) · `audit.rs` (79) · `scope.rs` (51) · `token.rs` (50) · `lib.rs` (47).

Tests: `adversarial.rs` (25 attacks) · `enforcement.rs` (5 scenarios).

Runtime dependencies (all public): `ed25519-dalek`, `base64`, `sha2`, `serde` / `serde_json`, `chrono`, `uuid`, `thiserror`, `axum`, `tokio`.

License: Apache-2.0. **Repo:** github.com/Polatkalender/citadel-mandate-demo